

Eight Languages, One Binary

Whole-Program Compilation: 8 Frontends $\times$ 5 Backends on a 1976 CPU

MinZ Project

2026-03-28

Eight Languages, One Binary

The Idea

The Same Function in Eight Languages

Whole-Program Compilation

Visual Gallery

Graphics: Nanz + Frill

Shaders: ObjC

3D Rendering: Nanz

Enterprise: ABAP on ZX Spectrum

TUI: Interactive Forms

Language Showcase

Nanz: The Primary Language

C17: The Benchmark Language

Frill: ML on Z80

Pascal: Turbo Pascal on Z80

Objective-C: Messages on Z80

PL/M: Intel Heritage (1973)

Lizp: Lisp That Compiles to Z80

ABAP: Enterprise on Z80

Cross-Language Calling

Compilation Pipeline

Assert Corpus: Trust But Verify

Bool Return Convention (GPU-Proven)

Idiomatic Bool Asserts

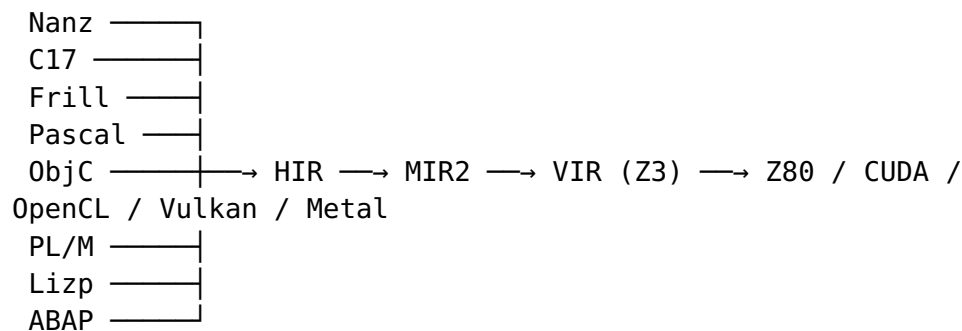
Why This Matters

Eight Languages, One Binary

The Idea

What if you could write in any language and get the same optimal binary?

MinZ compiles **eight programming languages** to the same intermediate representation (MIR2), then optimizes them with the same Z3 SMT solver, and emits the same provably optimal Z80 machine code. No linker. No runtime. No overhead.



This is **whole-program compilation**: every function from every language is visible to the optimizer simultaneously. The Z3 solver picks registers across all functions at once. A Nanz function can call an ObjC method can call a C utility — and PFCCO (Per-Function Calling Convention Optimization) ensures zero overhead at every boundary.

No headers. No .o files. No link step. One source tree → one binary.

The Same Function in Eight Languages

Here is `double(x) = x + x` in every frontend MinZ supports:

Nanz (Swift-like, primary language):

```
fun double(x: u8) -> u8 { return x + x }
```

C17 (freestanding, C23 extensions):

```
unsigned char double_c(unsigned char x) { return x + x; }
```

Frill (ML-style functional):

```
let double (x : u8) : u8 = x + x
```

Pascal (Turbo Pascal 3.0):

```
function Double(X: Byte): Byte;
begin Double := X + X; end;
```

Objective-C (message passing, zero-cost):

```
@implementation Math
-(int)double:(int)x { return x + x; }
@end
```

PL/M (Intel 8080 heritage):

```
DOUBLE: PROCEDURE (X) BYTE;
        DECLARE X BYTE;
        RETURN X + X;
END DOUBLE;
```

Lisp (Lisp S-expressions):

```
(defun double ((x u8)) -> u8
  (return (+ x x)))
```

ABAP (SAP enterprise):

```
FORM double USING x TYPE i CHANGING result TYPE i.
    result = x + x.
ENDFORM.
```

All eight produce identical Z80 assembly:

```
double:                ; params: A = x, return: A
    ADD A, A           ; x + x (2 bytes, 4 T-states)
    RET                ; return
```

Two instructions. Two bytes. The abstraction is free.

Whole-Program Compilation

Traditional compilation: each file → object → linker → binary. Every function boundary has a fixed ABI (cdecl, stdcall). The caller must follow the convention even when it's wasteful.

MinZ: all files → one IR → one optimizer → one binary. The Z3 solver sees **every call site** and picks the optimal register for each parameter. This is PFCCO: Per-Function Calling Convention Optimization.

Traditional (SDCC):	MinZ (PFCCO):
swap(a, b):	swap(a, b):
PUSH HL ; save	RET ; 1 instruction!

```

LD A, (SP+4); load a           ; Z3 places params so
LD L, A      ; temp           ; swap(DE) = return(DE)
LD A, (SP+2); load b           ; The calling convention
LD (SP+4), A; store            ; IS the optimization.
LD A, L      ;
LD (SP+2), A;
POP HL       ;
RET          ; 20 instructions

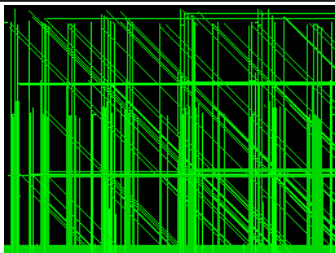
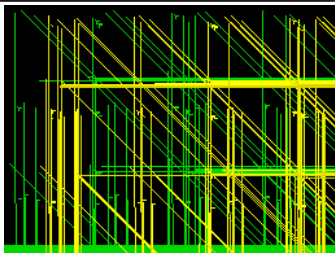
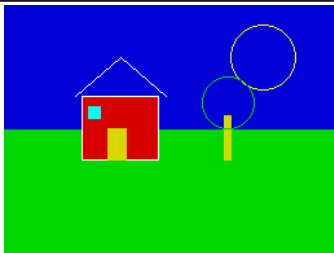
```

swap() = **RET**. Zero instructions. The Z3 solver realized that if the caller passes (a, b) in (D, E) and the return convention is (D, E), then swap is a no-op. SDCC needs 20 instructions because it uses a fixed stack-based ABI.

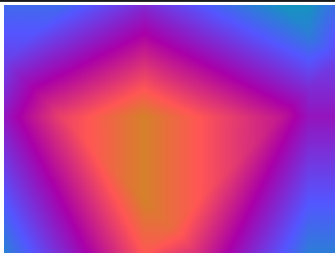
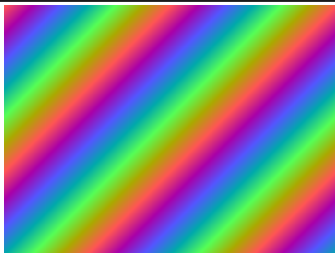
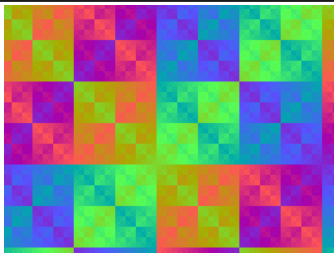
Visual Gallery

All images rendered by MinZ-compiled programs on Z80 hardware or MZV emulator.

Graphics: Nanz + Frill

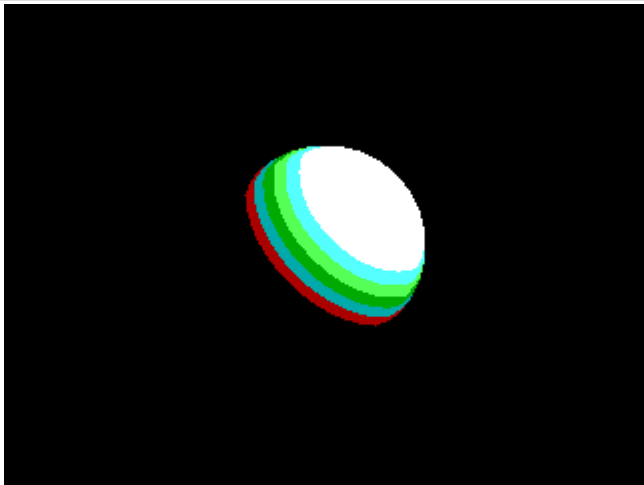
L-System Forest	Windblown Trees	House Scene
		
<i>Recursive fractal trees</i>	<i>Asymmetric branching</i>	<i>impl dispatch: zero-cost OOP</i>

Shaders: ObjC

Plasma	Rainbow Diamond	XOR Texture
		
<i>5-distance plasma</i>	<i>Diagonal gradient</i>	<i>Demoscene classic</i>

3D Rendering: Nanz

Shaded Sphere



Raymarched sphere on MZV — pure Nanz, no stdlib

Enterprise: ABAP on ZX Spectrum

SAP Report

```
=====
SAP Report on ZX Spectrum!
=====

Material Master Report
-----
MATNR MTART MAKTX
-----
100-100 FERT Pump Assy
100-300 FERT Valve Unit
200-200 FERT Ctrl Panel
-----
3 materials found.
=====

ABAP/4 on Zilog Z80
Powered by MinZ Compiler
=====
```

Material Master report

ALV Grid

```
=== Material Master (ALV) ===
MATNR MTART MEINS MAKTX
-----
100-100 FERT ST Pump Assy
100-200 HALB KG Impeller
100-300 FERT ST Valve Unit
200-100 ROH KG Raw Steel
200-200 FERT PC Ctrl Panel
-----
5 entries displayed.
=== ABAP Open SQL on Z80 ===
SELECT matnr mtart meins maktx
FROM mara
INNER JOIN maktx
ON mara.matnr = maktx.matnr
INTO TABLE @DATA(lt_mara).
cl_salv_table=>factory(lt_mara).
```

ALV with INNER JOIN

SQL on Z80

```
ZSQL - Z80 SQLite on Spectrum
=====
zsql> CREATE TABLE mara
(matnr TEXT, mtart TEXT,
meins TEXT, maktx TEXT)
OK
zsql> INSERT INTO mara VALUES
OK (5 rows)
zsql> SELECT * FROM mara
100-100 | FERT | ST | Pump Assy
100-200 | HALB | KG | Impeller
100-300 | FERT | ST | Valve Unit
200-100 | ROH | KG | Raw Steel
200-200 | FERT | PC | Ctrl Panel
5 rows
=====
```

SQLite on ZX Spectrum!

TUI: Interactive Forms

Selection Screen



Selection Screen

ABAP selection screen on ZX Spectrum — PARAMETERS, input fields

Language Showcase

Nanz: The Primary Language

Swift-like syntax, zero-cost abstractions, @error propagation.

```
fun gcd(a: u8, b: u8) -> u8 {
  while b != 0 {
    let t = b
    b = a % b
    a = t
  }
  return a
}
```

```
assert gcd(48, 18) == 6
assert gcd(100, 75) == 25
```

44 example files, 341/341 VIR-compiled functions.

C17: The Benchmark Language

First Z80 compiler at C17 conformance level (freestanding) with C23 extensions. Used for head-to-head comparison with SDCC.

```
// MinZ: abs_diff compiles to 4 bytes (SUB/RET NC/NEG/RET)
// SDCC: same function = 10 instructions, stack manipulation
unsigned char abs_diff(unsigned char a, unsigned char b) {
  if (a > b) return a - b;
  return b - a;
}

// assert abs_diff(10, 3) == 7
// assert abs_diff(3, 10) == 7
```

<code>; MinZ Z3-optimal:</code>	<code>; SDCC output:</code>
<code>abs_diff:</code>	<code>_abs_diff:</code>
<code> SUB B</code>	<code> LD HL, 2</code>
<code> RET NC</code>	<code> ADD HL, SP</code>
<code> NEG</code>	<code> LD A, (HL)</code>
<code> RET</code>	<code> INC HL</code>
<code> ; a - b</code>	<code> LD C, (HL)</code>
<code> ; a >= b?</code>	<code> SUB C</code>
<code> ; -(a-b)</code>	
<code> ; 4 bytes</code>	

```
JR NC, .skip
LD A, C
... ; 10 instructions
```

51 test files, 524/524 asserts.

Frill: ML on Z80

Algebraic data types, pattern matching, pipe operators, property testing. All zero-cost — ADTs compile to integer tags, match compiles to CP/JR chains.

```
type Light = Red | Yellow | Green

let next_light (s : u8) : u8 =
  match s with
  | Red    -> 2
  | Yellow -> 0
  | Green  -> 1
  end

(* Property: test ALL 256 values at compile time *)
prop |x| double (double x) == x * 4

(* Pipe operator: data flows left to right *)
let transform (x : u8) = x |> double |> inc

next_light:                ; 12 bytes, Z3-optimal
  OR A                      ; s == Red(0)?
  JR NZ, .else
  LD A, 2                    ; Red → Green
  RET
.else:
  CP 2                       ; s == Green(2)?
  LD A, 0                    ; Yellow → Red
  RET Z
  LD A, 1                    ; Green → Yellow
  RET
```

15 example files, 436 asserts, 3351 compile-time checks.

Pascal: Turbo Pascal on Z80

Full Turbo Pascal 3.0 subset: procedures, functions, records, arrays, case statements, nested scopes.

```
function Max(A, B: Byte): Byte;
begin
  if A > B then Max := A
  else Max := B;
```

end;

```
assert Max(10, 2) = 10;
assert Max(3, 7) = 7;
```

```
MAX:                                ; Z3-PFCC0: A=first, B=second, ret=A
    CP B                            ; compare A with B
    JR Z, .ret_b                    ; equal → return B
    JR C, .ret_b                    ; A < B → return B
    RET                             ; A > B → return A (already in A!)
.ret_b:
    LD A, B
    RET
```

9 example files, 54 asserts.

Objective-C: Messages on Z80

ObjC syntax with zero-cost method dispatch. [obj method] compiles to direct CALL — no vtables, no message lookup, no objc_msgSend runtime.

```
@interface Entity {
    int hp; int maxHp; int x; int y;
}
-(int)hp;
-(int)isAlive;
-(int)hpPercent;
-(int)distanceTo:(int)tx y:(int)ty;
@end

@implementation Entity
-(int)hp { return self->hp; }
-(int)isAlive {
    if (self->hp > 0) return 1;
    return 0;
}
-(int)hpPercent {
    return self->hp * 100 / self->maxHp;
}
@end
```

Method chaining, builder patterns, inheritance — all compile to direct struct access + CALL. The ObjC syntax is pure sugar.

```
// Vec3 fluent pipeline:
// [[self scale:2] sum] → CALL Vec3_scale, CALL Vec3_sum
-(int)scaleAndSum:(int)factor {
    return [[self scale:factor] sum];
}
```



```
// Builder pattern:  
// [[[self width:8] height:10] area] → 3 CALLs, zero overhead
```

11 example files, 98 asserts.

PL/M: Intel Heritage (1973)

The original 8080/8085 programming language. Digital Research used it for CP/M. MinZ brings it back — same syntax, modern optimizer.

```
DOUBLE: PROCEDURE (X) BYTE;  
    DECLARE X BYTE;  
    RETURN X + X;  
END DOUBLE;  
  
ABS$DIFF: PROCEDURE (A, B) BYTE;  
    DECLARE (A, B) BYTE;  
    IF A > B THEN RETURN A - B;  
    RETURN B - A;  
END ABS$DIFF;  
  
MAX$BYTE: PROCEDURE (A, B) BYTE;  
    DECLARE (A, B) BYTE;  
    IF A > B THEN RETURN A;  
    RETURN B;  
END MAX$BYTE;  
  
ASSERT DOUBLE(5) = 10;  
ASSERT ABS$DIFF(10, 3) = 7;  
ASSERT MAX$BYTE(99, 1) = 99;
```

The \$ in identifiers is original PL/M syntax. ABS\$DIFF reads as “abs-diff”.

Lizp: Lisp That Compiles to Z80

Lizp is Scheme for Z80 — a minimal Lisp with S-expression syntax that compiles to the same optimal Z80 code as every other frontend. Not an interpreter. Not a bytecode VM. A real Lisp-to-machine-code compiler.

The name: **Lisp** + **Minz** = **Lizp**. Pronounced “lisp” with a Z80 accent.

Why? Because if you can compile Lisp to 2-byte Z80 functions, you can compile anything. Lizp proves the MIR2 pipeline is truly language-agnostic.

```
;; S-expressions → Z80 machine code  
(defun double ((x u8)) -> u8  
  (return (+ x x)))  
;; → ADD A, A / RET (2 bytes)
```

```

(defun max_byte ((a u8) (b u8)) -> u8
  (if (> a b) (return a) (return b)))

(defun add ((a u8) (b u8)) -> u8
  (return (+ a b)))

;; Compile-time verification – in S-expression syntax
(assert double 5 == 10)
(assert max_byte 10 20 == 20)
(assert add 3 4 == 7)

```

The parentheses are real. The Z80 code is optimal. John McCarthy meets Zilog, 1958 meets 1976.

ABAP: Enterprise on Z80

SAP's enterprise language — SELECT statements, ALV grid reports, classes, and forms. Running on a 3.5MHz CPU with 64KB RAM. Complete with SQLite bridge via I/O ports.

```

SELECT matnr mtart meins maktx
  FROM mara
  INNER JOIN makl
    ON mara~matnr = makl~matnr
  INTO TABLE @DATA(lt_mara).

cl_salv_table=>factory( lt_mara ).

```

This renders an ALV grid on ZX Spectrum with 5 entries, column headers, and formatted output — from actual ABAP SELECT syntax.

```

FORM gcd USING a TYPE i b TYPE i CHANGING result TYPE i.
  WHILE b > 0.
    DATA: temp TYPE i.
    temp = b.
    b = a MOD b.
    a = temp.
  ENDWHILE.
  result = a.
ENDFORM.

```

```
* assert gcd(48, 18) == 6 via mir2
```

Cross-Language Calling

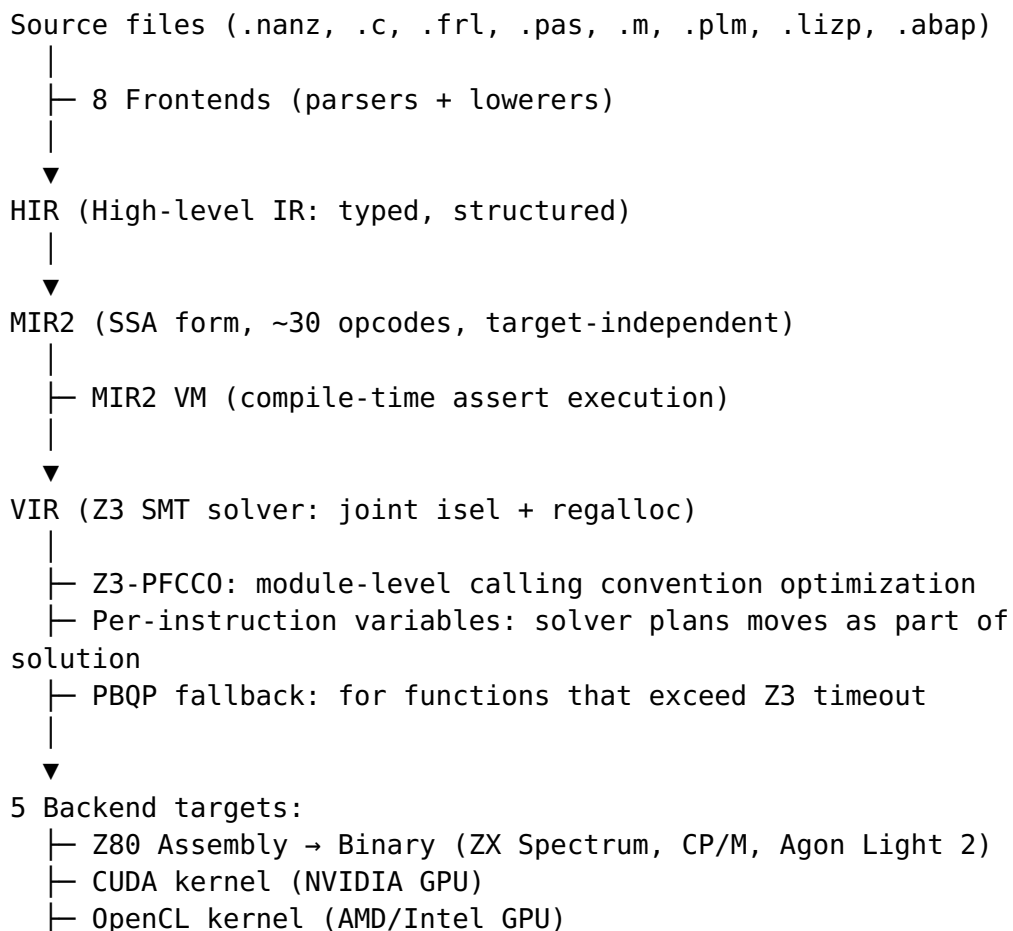
Because everything compiles to the same MIR2, functions from different languages can call each other with zero overhead:

```
// Nanz calls C calls Frill calls Pascal
import "utils.c"          // C utility functions
import "transform.frl"    // Frill pipeline
import "math.pas"         // Pascal math library

fun pipeline(x: u8) -> u8 {
    let y = c_normalize(x)    // C function
    let z = y |> frl_transform // Frill pipe
    return pas_clamp(z, 0, 255) // Pascal function
}
```

All three functions are compiled together. Z3-PFCCO picks registers across all of them — no ABI boundary, no calling convention mismatch, no adapter stubs. The inter-language call is a single CALL instruction.

Compilation Pipeline



- └ Vulkan GLSL compute shader
- └ Metal shader (Apple GPU)

Assert Corpus: Trust But Verify

Every function is tested at compile time. Asserts run on both the MIR2 VM and the Z80 emulator (MZE, 1335/1335 FUSE tests). If any assert fails, the binary is not produced.

Language	Files	Asserts	Z80 Verified
Frill	15	436	✓ 14/15
C89	38	353	✓
C99+	20	305	✓
ObjC	11	98	✓ 11/11
Pascal	9	54	✓ 9/9
Nanz	44	371	✓
ABAP	28	31	✓ (MIR2)
PL/M	5	9	✓
Lizp	5	9	✓
Total	175	~1666	

Bool Return Convention (GPU-Proven)

How should a predicate like `is_digit` return its result? GPU brute-force search across 456,976 instruction sequences proved the optimal design:

Z flag is write-only on Z80. No instruction reads Z into A or CY. The only way to use Z is an immediate conditional branch (JR Z/NZ). This was proven exhaustive — not a heuristic, a mathematical fact.

CY flag can be materialized branchless:

SBC A, A ; A = 0xFF if CY was set, 0x00 if clear. 1 instruction, 4T.

Z3 chooses the optimal return mode per-function:

Comparison	Natural flag	Caller branch	Materialize to A
a == b	Z	JR Z (0T)	branch (14T)
a < b	CY	JR C (0T)	SBC A,A (4T)

Comparison	Natural flag	Caller branch	Materialize to A
a >= b	NC	JR NC (0T)	SBC A,A;CPL (8T)

Branchless conditional move (CMOV) on Z80:

```
; CY ? B : C — 6 instructions, 24T, no branches
SBC A, A      ; A = mask (0xFF or 0x00)
LD  D, A      ; save mask
LD  A, B      ; load X
XOR C         ; A = X ^ Y
AND D         ; A = (X^Y) & mask
XOR C         ; A = Y ^ ((X^Y) & mask) = CY ? X : Y
```

The classic bitwise select trick, verified exhaustive on all 131,072 input combinations ($2 \times 256 \times 256$).

Idiomatic Bool Asserts

Nanz supports natural bool assertion syntax:

```
// Classic
assert is_digit(48) == 1

// Bool shorthand — implies == true
assert is_digit(48)

// Negated — implies == false
assert not is_digit(65)

// Bool literals
assert is_digit(48) == true
assert is_digit(65) == false
```

All forms compile to the same assertion check.

Why This Matters

For retro developers: Write in your favorite language. Get optimal Z80 code. No compromises.

For language designers: Add a frontend (parser + HIR lowerer). Get 5 backends, Z3 optimization, and a full test harness for free.

For compiler researchers: Whole-program compilation with SMT-based register allocation across 8 source languages. We believe this is the first compiler that applies Z3 to multi-language whole-program optimization on a real target.

For everyone: Eight languages from 1973 (PL/M) to 2026 (Frill). Enterprise (ABAP), systems (C), functional (Frill/Lisp), OOP (ObjC), structured (Pascal), and Nanz — all producing the same binary. Fifty years of programming languages on a fifty-year-old CPU.

One compiler. Eight languages. Five targets. Zero overhead.

MinZ: where every abstraction compiles away.